

Extraclase 2: CI/CD con GitHub Actions

Informe de evidencias y análisis comparativo

Estudiante: Dorling Tamara Mora Martínez

Universidad Latina de Costa Rica

Programación IV

Trabajo Extraclase — Valor: 5%

II Ciclo, 2026

Repositorio: <https://github.com/dorlingm07/Extraclase-2-p4>

Introducción

En este documento presento el trabajo que realicé para el Extraclase 2 del curso Programación IV, sobre integración y despliegue continuo con GitHub Actions. Para la parte práctica desarrollé una API REST sencilla con Node.js y Express, que me sirvió como caso de estudio para construir tres workflows distintos: uno de integración continua, uno de despliegue continuo hacia Vercel y uno de pruebas en matriz sobre varias versiones de Node.js y sistemas operativos. También incluyo, como se pide en el Ejercicio 4, un análisis comparativo entre GitHub Actions y otras dos plataformas de CI/CD.

El código completo del proyecto, junto con los tres archivos de workflow, se encuentra en el repositorio público de GitHub que indico en la portada de este documento. En las siguientes secciones documento lo que implementé para cada ejercicio y dejo marcado el espacio donde debo insertar las capturas de pantalla que evidencian que los pipelines corrieron correctamente.

Ejercicio 1: Pipeline de CI para la aplicación web

Para este ejercicio construí una API de gestión de tareas con Express (archivo `src/app.js`), con endpoints para crear, listar, actualizar y eliminar tareas. Separé la lógica de negocio en el archivo `src/tareas.js` para poder probarla de forma unitaria sin depender del servidor HTTP.

El workflow de integración continua está definido en `.github/workflows/ci.yml` y cumple con todos los requisitos que pedía el enunciado:

- Se dispara con push a las ramas `main` y `develop`, y con pull requests hacia `main`.
- Incluye un paso de linting con ESLint (`npm run lint`) que revisa el estilo del código.
- Ejecuta 17 pruebas unitarias y de integración con Jest y Supertest (bastante más de las 5 mínimas pedidas), que cubren la lógica de `tareas.js` y los endpoints de `app.js`.

- Genera un reporte de cobertura de código con Jest y lo sube como artifact del workflow (coverage-report).
- Agrega una notificación de éxito o fallo del pipeline en el resumen del job (GITHUB_STEP_SUMMARY), además de los status checks nativos de GitHub.

The screenshot displays the GitHub Actions interface for a CI Pipeline. At the top, the title 'CI Pipeline' is followed by a link to 'ci.yml'. A search bar labeled 'Filter workflow runs' and a menu icon are also present. Below this, it indicates '2 workflow runs'. A table-like structure shows two runs, both with a green checkmark icon. The first run is for the 'develop' branch, triggered by a commit push, and completed in 19 seconds. The second run is for the 'main' branch, also triggered by a commit push, and completed in 22 seconds. Below the runs, there is a section titled 'build-and-test summary' which contains a large green checkmark and the text 'CI exitoso: el build, el lint y las pruebas pasaron correctamente.' followed by a link to the job summary.

CI Pipeline
[ci.yml](#)

Filter workflow runs

2 workflow runs

Event	Status	Branch	Actor	Time
✓	Agrego el README y el inform...	develop	dorlingm07	Today at 1:04 AM 19s
✓	Agrego el README y el inform...	main	dorlingm07	Today at 1:04 AM 22s

build-and-test summary

CI exitoso: el build, el lint y las pruebas pasaron correctamente.

[Job summary generated at run-time](#)

Artifacts			
Produced during runtime			
Name	Size	Digest	
 coverage-report	22.9 KB	sha256:daa889c6aa3f9b63fb9c84da0fab6d168db870ced...	 

Ejercicio 2: Pipeline de CD con despliegue automático

El workflow de despliegue está definido en `.github/workflows/cd.yml`. Se dispara automáticamente cuando el workflow de CI termina exitosamente sobre la rama main (usando el evento `workflow_run`), construye el proyecto y lo despliega a Vercel usando la CLI oficial de Vercel dentro del propio job.

Configuré el job de despliegue para que use un environment de GitHub llamado `production`, y los tokens de Vercel (`VERCEL_TOKEN`, `VERCEL_ORG_ID` y `VERCEL_PROJECT_ID`) se leen desde GitHub Secrets, nunca quedan escritos en el código. Para que este pipeline despliegue de verdad, en el repositorio de GitHub debo completar dos configuraciones manuales que no se pueden hacer desde el código: crear el environment "production" en Settings > Environments con una regla de protección (por ejemplo, un aprobador requerido), y registrar los tres secretos de Vercel en Settings > Secrets and variables > Actions.

← CD Pipeline

✓

CD Pipeline #2

Re-run all jobs

...

Summary

All jobs

✓ deploy

Run details

Usage

Workflow file

Triggered via workflow run 26 minutes ago

Status

Total duration

Artifacts

 **dorlingm07** completed  [a4f8f92](#)

Success

23m 16s

—

cd.yml

on: workflow_run

✓ deploy 35s

<https://extraclase2p4-fh2w48fsu-dorlingta...>

⌵

—

+

deploy summary

...

Despliegue exitoso a produccion: <https://extraclase2p4-fh2w48fsu-dorlingtamara-9993s-projects.vercel.app>

[Job summary generated at run-time](#)

Environments / Configure production

Deployment protection rules

Configure reviewers, timers, and custom rules that must pass before deployments to this environment can proceed.

☒ **Required reviewers**

Specify people or teams that may approve workflow runs when they access this environment.

Add up to 5 more reviewers

Search for people or teams...



dorlingm07



☐ **Prevent self-review**

Require a different approver than the user who triggered the workflow run.

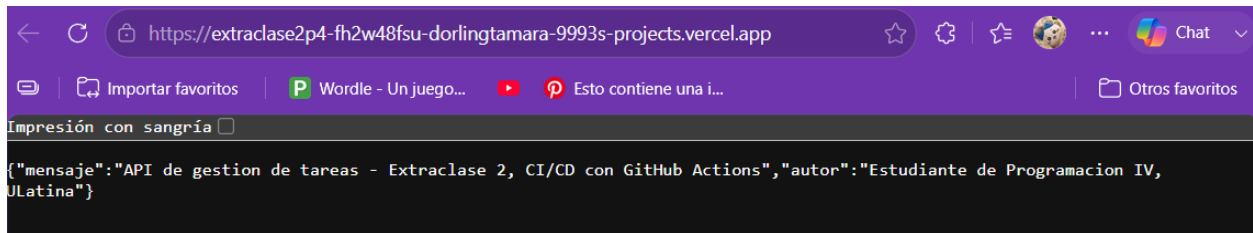
☐ **Wait timer**

Set an amount of time to wait before allowing deployments to proceed.

Enable custom rules with GitHub Apps Preview

[Learn about existing apps](#) or [create your own protection rules](#) so you can deploy with confidence.

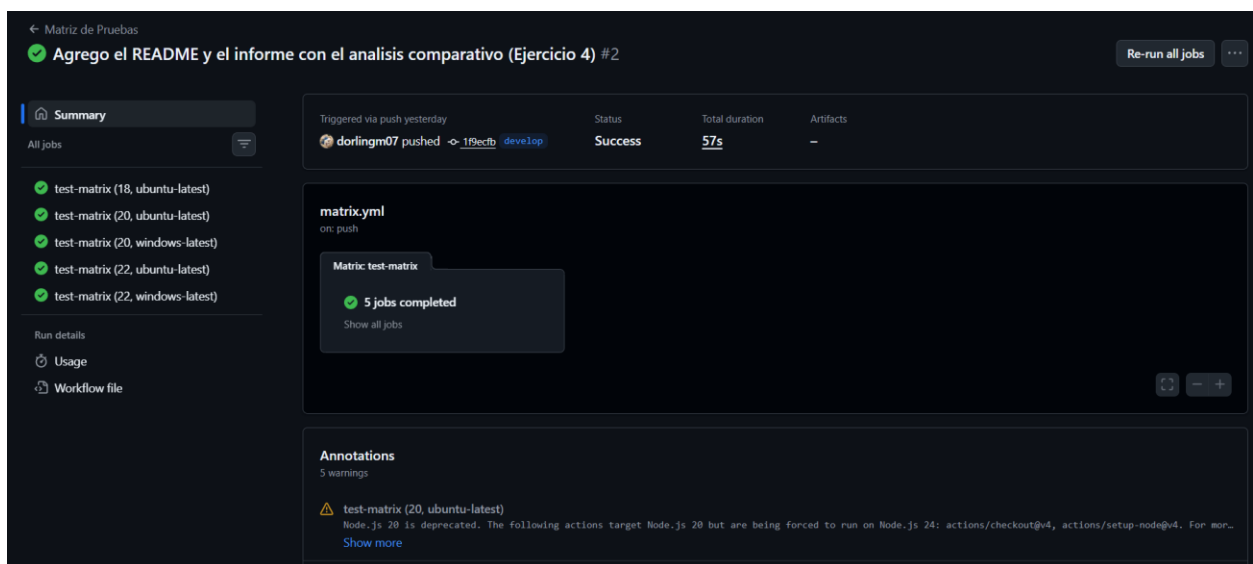
Name 	Last updated		
 VERCEL_ORG_ID	32 minutes ago		
 VERCEL_PROJECT_ID	32 minutes ago		
 VERCEL_TOKEN	33 minutes ago		



<https://extracase2p4-fh2w48fsu-dorlingtamara-9993s-projects.vercel.app>

Ejercicio 3: Workflow avanzado con matriz de pruebas

El workflow `.github/workflows/matrix.yml` usa la estrategia de matriz de GitHub Actions para correr las pruebas en varias combinaciones de versión de Node.js y sistema operativo al mismo tiempo. La matriz combina tres versiones de Node (18, 20 y 22) con dos sistemas operativos (ubuntu-latest y windows-latest), usa `fail-fast: false` para que todas las combinaciones terminen de ejecutarse aunque alguna falle, y excluye la combinación de Node 18 con windows-latest a modo de ejemplo del uso de `exclude` que pedía el enunciado.



Ejercicio 4: Análisis comparativo de plataformas CI/CD

Para este ejercicio decidí comparar GitHub Actions con otras dos plataformas de CI/CD ampliamente usadas en la industria: GitLab CI/CD y Jenkins. Elegí estas dos porque representan enfoques bastante distintos al mismo problema: GitLab CI/CD es, igual que GitHub Actions, una solución integrada dentro de una plataforma de control de versiones, mientras que Jenkins es una herramienta independiente, de código abierto y autohospedada que existe desde mucho antes de que el concepto de CI/CD se popularizara.

En cuanto al modelo de precios y plan gratuito, GitHub Actions ofrece 2,000 minutos mensuales gratuitos para repositorios privados en el plan Free, y minutos ilimitados para repositorios públicos, lo cual fue clave para este trabajo porque pude ejecutar los tres workflows sin ningún costo. GitLab CI/CD tiene un esquema parecido, con 400 minutos gratuitos al mes para proyectos privados y minutos ilimitados en proyectos públicos, aunque su límite gratuito para proyectos privados es notablemente menor que el de GitHub. Jenkins, por su parte, es gratuito porque es software libre, pero ese "gratis" es un poco engañoso: como es autohospedado, hay que asumir el costo de la infraestructura donde corre (un servidor o una máquina virtual en la nube) y el tiempo que toma mantenerlo actualizado y seguro.

Sobre la facilidad de configuración y la curva de aprendizaje, esta fue quizás la diferencia que más sentí mientras trabajaba en este extraclase. Con GitHub Actions me bastó con crear un archivo YAML dentro de `.github/workflows/` y el pipeline quedó funcionando sin instalar nada adicional ni configurar ningún servidor propio. GitLab CI/CD funciona de forma muy parecida, con un único archivo `.gitlab-ci.yml` en la raíz del proyecto, así que la curva de aprendizaje es comparable si ya se conoce YAML. Jenkins es bastante más complejo: hay que instalar el propio Jenkins (normalmente con Docker o en un servidor), configurar plugins, crear jobs desde su interfaz web o mediante un Jenkinsfile en Groovy, y administrar los propios agentes o runners.

Para un estudiante o un equipo pequeño que solo quiere automatizar pruebas rápidamente, esa complejidad adicional de Jenkins se siente injustificada; donde sí tiene sentido es en organizaciones grandes que ya cuentan con un equipo de DevOps dedicado a mantenerlo.

En cuanto al ecosistema de plugins y acciones reutilizables, GitHub Actions tiene a su favor el Marketplace, con miles de acciones creadas tanto por GitHub como por la comunidad, como las que usé en este trabajo (actions/checkout, actions/setup-node, actions/upload-artifact). GitLab CI/CD no tiene un "marketplace" como tal, sino un catálogo de componentes de CI/CD y plantillas incluidas, que es funcional pero menos extenso que el de GitHub. Jenkins, en cambio, tiene el ecosistema de plugins más grande y maduro de los tres, con más de 1,800 plugins disponibles, resultado de más de una década de existencia, aunque en la práctica muchos de esos plugins están desactualizados o mal mantenidos.

Los tres tienen buen soporte para contenedores y Docker. GitHub Actions permite ejecutar jobs dentro de contenedores y construir o publicar imágenes fácilmente, por ejemplo con la acción docker/build-push-action que se menciona en la parte teórica de este trabajo. GitLab CI/CD tiene una integración de Docker particularmente pulida, con su propio Container Registry integrado en cada proyecto. Jenkins también soporta Docker mediante plugins, aunque en general requiere más configuración manual que las otras dos opciones.

Respecto a la integración con proveedores de nube, las tres plataformas se integran razonablemente bien con AWS, Azure y GCP, principalmente a través de acciones o plugins oficiales y del uso de tokens guardados como secretos. En mi caso, para el Ejercicio 2 usé GitHub Actions junto con la CLI de Vercel y GitHub Secrets para desplegar automáticamente, y el proceso fue bastante directo.

Finalmente, en cuanto al soporte para monorepos y proyectos de gran escala, GitLab CI/CD tiene una ventaja real gracias a sus reglas de `rules/only/except` y a la posibilidad de definir pipelines padre-hijo pensadas específicamente para monorepos grandes. GitHub Actions puede lograr algo similar usando filtros de paths y workflows reutilizables, pero requiere más configuración manual. Jenkins, al ser tan flexible, también puede adaptarse a monorepos, aunque de nuevo a costa de bastante configuración propia.

Mi recomendación, después de trabajar directamente con GitHub Actions en este extracase, es que para proyectos que ya viven en GitHub, como el mío, tiene mucho sentido usar GitHub Actions, porque no hay que pagar ni mantener infraestructura aparte y la curva de aprendizaje es baja. Si un equipo ya trabaja en GitLab, la recomendación lógica sería usar GitLab CI/CD por esa misma razón de integración nativa. Jenkins lo recomendaría solamente para organizaciones grandes con necesidades muy particulares, por ejemplo builds que requieren hardware específico, acceso a redes internas restringidas o integraciones con sistemas legados, que sí justifiquen tener un equipo dedicado a mantenerlo.

Conclusión general

Hacer este extracase me ayudó a entender CI/CD no solo en la teoría, sino aplicándolo directamente sobre un proyecto propio. Configurar los tres workflows me hizo ver con más claridad por qué la automatización de pruebas y despliegues reduce tanto el margen de error humano, y también me dejó más clara la diferencia práctica entre entrega continua y despliegue continuo al tener que decidir cómo proteger el environment de producción. El análisis comparativo del último ejercicio, además, me sirvió para justificar con criterios concretos por qué GitHub Actions fue la herramienta más conveniente para este caso particular.

Referencias

GitHub. (s.f.). GitHub Actions Documentation. <https://docs.github.com/en/actions>

GitHub. (s.f.). Understanding GitHub Actions. <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>

GitHub. (s.f.). Workflow syntax for GitHub Actions. <https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions>

GitLab. (s.f.). GitLab CI/CD Documentation. <https://docs.gitlab.com/ee/ci/>

Jenkins. (s.f.). Jenkins User Documentation. <https://www.jenkins.io/doc/>

Vercel. (s.f.). Vercel Documentation. <https://vercel.com/docs>

Humble, J. y Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.

Fowler, M. (2006). Continuous Integration.

<https://martinfowler.com/articles/continuousIntegration.html>

Kim, G., Humble, J., Debois, P. y Willis, J. (2016). The DevOps Handbook. IT Revolution Press.